

OOP Principles: Encapsulation, Inheritance, Polymorphism & Abstraction

A good mental model:

```
OOP
|
├─ Encapsulation → Protect/manage state
├─ Abstraction   → Expose what matters, hide details
├─ Inheritance   → Reuse/extend behavior
└─ Polymorphism  → Same interface, different behavior
```

1. Encapsulation

What is encapsulation?

Encapsulation means:

Bundling data and the operations that manipulate that data together, while controlling direct access to the internal state.

Classic Java example:

```
public class BankAccount {

    private BigDecimal balance;

    public void deposit(BigDecimal amount) {
        if (amount.signum() <= 0) {
            throw new IllegalArgumentException("Invalid amount");
        }

        balance = balance.add(amount);
    }

    public BigDecimal getBalance() {
        return balance;
    }
}
```

The important part:

```
private BigDecimal balance;
```

External code cannot directly do:

```
account.balance = -5000; // ✗
```

Instead:

```
account.deposit(amount);
```

The class controls how its state changes.

Why is encapsulation useful?

It protects **invariants**.

Suppose a `Payment` object has:

```
status = PENDING
```

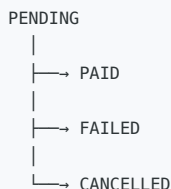
You don't want arbitrary code doing:

```
payment.status = "RANDOM_VALUE";
```

Instead, expose controlled behavior:

```
payment.markPaid();
payment.markFailed();
payment.cancel();
```

Now the entity/service can enforce valid transitions.



This is much stronger than simply making every field public.

2. Encapsulation in Spring Boot

A common service:

```
@Service
public class PaymentService {

    private final PaymentRepository repository;

    public PaymentService(PaymentRepository repository) {
        this.repository = repository;
    }

    public Payment process(Long paymentId) {
        Payment payment = repository.findById(paymentId)
            .orElseThrow();
    }
}
```

```

        validate(payment);
        updateStatus(payment);

        return payment;
    }

    private void validate(Payment payment) {
        // validation logic
    }

    private void updateStatus(Payment payment) {
        // state transition logic
    }
}

```

The caller doesn't need to know:

```

How validation works
How status is changed
How repository is accessed

```

It only knows:

```

paymentService.process(id);

```

That's encapsulation of implementation details.

3. Abstraction

Abstraction means:

Expose the essential behavior while hiding implementation details.

For example:

```

public interface PaymentProcessor {

    PaymentResult process(PaymentRequest request);
}

```

The caller knows:

```

"process a payment"

```

but doesn't need to know how the provider actually processes it.

Implementation:

```

public class HydraPaymentProcessor
    implements PaymentProcessor {

    @Override
    public PaymentResult process(PaymentRequest request) {
        // Hydra-specific implementation
    }
}

```

Another implementation:

```
public class ChasePaymentProcessor
    implements PaymentProcessor {

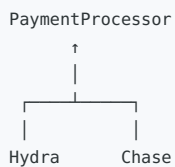
    @Override
    public PaymentResult process(PaymentRequest request) {
        // Chase-specific implementation
    }
}
```

Caller:

```
PaymentProcessor processor = ...;

processor.process(request);
```

The caller depends on the abstraction:



4. Abstraction vs Encapsulation

These are commonly confused.

Abstraction

Focuses on:

What should the object expose?

Example:

```
interface PaymentProcessor {
    PaymentResult process(PaymentRequest request);
}
```

Encapsulation

Focuses on:

How do we protect/hide the internal state and implementation?

Example:

```
private PaymentStatus status;
```

Mental shortcut:

Abstraction
→ What?

Encapsulation
→ How is the internal state protected/hidden?

They often work together.

5. Abstract Class

Java supports abstraction through abstract classes too.

```
public abstract class PaymentProcessor {  
  
    public abstract PaymentResult process(  
        PaymentRequest request);  
  
    protected void logPayment(PaymentRequest request) {  
        System.out.println("Payment received");  
    }  
}
```

Implementation:

```
public class HydraProcessor extends PaymentProcessor {  
  
    @Override  
    public PaymentResult process(PaymentRequest request) {  
        // Hydra implementation  
    }  
}
```

Use an abstract class when you have:

Common state
+
Common implementation
+
Some behavior that subclasses must implement

6. Interface vs Abstract Class

Interface	Abstract Class
Defines a contract/capability	Can define common base behavior
A class can implement multiple interfaces	A class can extend only one class
Excellent for loose coupling	Useful for shared implementation/state
No instance state in the traditional sense	Can have instance fields
Can contain default/static/private methods in modern Java	Can contain normal concrete methods

For backend design, interfaces are often used at boundaries:

```
Service abstraction
Payment provider abstraction
Notification abstraction
Storage abstraction
```

7. Inheritance

Inheritance means:

A class derives behavior/state from another class.

Example:

```
public class PaymentException extends RuntimeException {
}
```

Here:

```
PaymentException
  ↓ extends
RuntimeException
```

It inherits properties/behavior from the parent class.

Another example:

```
public class Vehicle {
    public void start() {
        System.out.println("Starting");
    }
}

public class Car extends Vehicle {
}
```

Now:

```
Car car = new Car();
car.start();
```

`Car` inherits `start()` .

8. Why Use Inheritance?

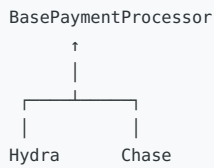
It can provide:

Code reuse

Common behavior can live in the parent.

Specialization

Child classes can add/override behavior.



But inheritance should be used carefully.

9. Composition vs Inheritance

Modern backend design often prefers:

Composition over inheritance

Suppose:

```
public class PaymentService {

    private final PaymentValidator validator;
    private final PaymentRepository repository;

    public PaymentService(
        PaymentValidator validator,
        PaymentRepository repository) {
        this.validator = validator;
        this.repository = repository;
    }
}
```

`PaymentService` has a:

```
PaymentValidator
PaymentRepository
```

This is composition.

Compare:

```
Inheritance
Car IS-A Vehicle

Composition
PaymentService HAS-A PaymentValidator
```

10. When Inheritance Becomes a Problem

Suppose:

```
BasePaymentProcessor
  ↓
OnlinePaymentProcessor
  ↓
HydraPaymentProcessor
  ↓
SpecialHydraPaymentProcessor
```

Now behavior becomes tightly coupled to the inheritance tree.

Changing the base class can affect many subclasses.

This is often called **fragile base-class behavior**.

Composition allows you to change components more independently.

11. Polymorphism

Polymorphism means:

The same interface/reference can represent different concrete implementations, with behavior determined at runtime.

Example:

```
PaymentProcessor processor;

processor = new HydraPaymentProcessor();
processor.process(request);

processor = new ChasePaymentProcessor();
processor.process(request);
```

Same method:

```
process()
```

Different implementation.

```
PaymentProcessor
├── HydraPaymentProcessor
│   └── → Hydra implementation
└── ChasePaymentProcessor
    └── → Chase implementation
```

That is runtime polymorphism.

12. Method Overriding = Runtime Polymorphism

```
class Animal {
```

```

    public void makeSound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {

    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {

    @Override
    public void makeSound() {
        System.out.println("Meow");
    }
}

```

Then:

```

Animal animal = new Dog();

animal.makeSound();

```

Output:

```
Bark
```

Even though the reference type is:

```
Animal
```

the actual object is:

```
Dog
```

So the overridden implementation is selected at runtime.

13. Compile-Time vs Runtime Polymorphism

Compile-time polymorphism

Usually method overloading.

```

public void process(Payment payment) {}

public void process(Payment payment,
                  boolean retry) {}

```

Compiler decides which method to call based on the method signature.

Runtime polymorphism

Method overriding:

```
PaymentProcessor p =  
    new HydraPaymentProcessor();  
  
p.process(request);
```

Actual implementation is selected at runtime.

For interviews:

```
Overloading → compile time  
Overriding → runtime
```

14. Real-World Usage: Strategy Pattern

This is one of the best examples because it combines:

```
Abstraction  
+  
Polymorphism  
+  
Encapsulation  
+  
Composition
```

Suppose different payment vendors behave differently.

Define:

```
public interface PaymentProcessor {  
  
    PaymentResult process(PaymentRequest request);  
}
```

Implement:

```
public class HydraPaymentProcessor  
    implements PaymentProcessor {  
  
    @Override  
    public PaymentResult process(PaymentRequest request) {  
        // Hydra-specific processing  
    }  
}
```

```
public class ChasePaymentProcessor  
    implements PaymentProcessor {  
  
    @Override  
    public PaymentResult process(PaymentRequest request) {  
        // Chase-specific processing  
    }  
}
```

Then:

```
public class PaymentService {
```

```

private final PaymentProcessor processor;

public PaymentService(PaymentProcessor processor) {
    this.processor = processor;
}

public PaymentResult process(
    PaymentRequest request) {
    return processor.process(request);
}
}

```

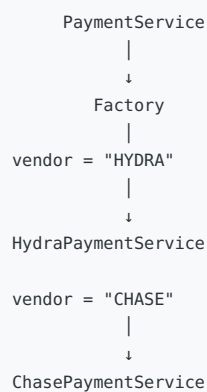
The service doesn't care which vendor implementation it received.
That's polymorphism.

15. Your BPS Example

Your BPS payment flow is a very good real-world example of this design.

You have a `vendor` value in the payment token, and BPS uses **Strategy + Factory** to select/create the appropriate vendor-specific service.

Conceptually:



The common abstraction could conceptually be:

```

public interface VendorPaymentService {

    PaymentResult process(PaymentRequest request);
}

```

Implementations:

```

class HydraPaymentService
    implements VendorPaymentService {

    @Override
    public PaymentResult process(PaymentRequest request) {
        // Hydra-specific logic
    }
}

```

```

class ChasePaymentService
    implements VendorPaymentService {

```

```

@Override
public PaymentResult process(PaymentRequest request) {
    // Chase-specific logic
}
}

```

Factory:

```

public class VendorPaymentFactory {

    public VendorPaymentService getService(
        String vendor) {

        return switch (vendor) {
            case "HYDRA" -> new HydraPaymentService();
            case "CHASE" -> new ChasePaymentService();
            default -> throw new IllegalArgumentException(
                "Unsupported vendor");
        };
    }
}

```

Then:

```

VendorPaymentService service =
    factory.getService(token.getVendor());

service.process(request);

```

Which OOP principles are involved?

```

Abstraction
↓
VendorPaymentService interface

Polymorphism
↓
Hydra / Chase implementations

Encapsulation
↓
Each vendor hides its own implementation details

Composition
↓
PaymentService uses Factory/VendorPaymentService

```

This is a much stronger interview example than saying:

"I used inheritance in my project."

16. Real-World Example: Notification System

Suppose an application supports:

```

Email
SMS
Push

```

Abstraction:

```
public interface NotificationSender {  
  
    void send(String message);  
}
```

Implementations:

```
class EmailSender implements NotificationSender {  
    public void send(String message) {  
        // SMTP/API  
    }  
}
```

```
class SmsSender implements NotificationSender {  
    public void send(String message) {  
        // SMS provider  
    }  
}
```

Caller:

```
NotificationSender sender = getSender();  
  
sender.send("Payment processed");
```

The caller doesn't care whether the implementation is:

```
Email  
SMS  
Push
```

That's abstraction + polymorphism.

17. Real-World Example: Payment Status

Encapsulation can be applied to business rules.

Bad:

```
payment.setStatus("PAID");
```

everywhere in the application.

Better:

```
payment.markPaid();
```

Inside:

```
public void markPaid() {  
  
    if (status != PaymentStatus.PENDING) {
```

```

        throw new IllegalStateException(
            "Only pending payment can be marked paid");
    }

    status = PaymentStatus.PAID;
}

```

Now the entity protects its invariant.

That's **real encapsulation**, not just "make fields private."

18. Real-World Example: Repository Abstraction

Your service might depend on:

```

public interface PaymentRepository {
    Payment findById(Long id);
    Payment save(Payment payment);
}

```

The service doesn't need to know whether storage is:

```

MySQL
PostgreSQL
MongoDB
Mock
External service

```

The service depends on an abstraction.

Spring Data takes this even further with:

```

public interface PaymentRepository
    extends JpaRepository<Payment, Long> {
}

```

Your business code uses repository operations without implementing JDBC/SQL connection management itself.

19. Real-World Example: Logging Abstraction

Your application can use:

```

private static final Logger log =
    LoggerFactory.getLogger(PaymentService.class);

```

Your service doesn't care about:

```

console appender
file appender
JSON logging
Splunk ingestion

```

The logging API abstracts the actual logging implementation.

This is another everyday example of abstraction in backend systems.

20. Why OOP Matters in Microservices

Imagine a Payment Service:

```
PaymentController
  ↓
PaymentService
  ↓
PaymentProcessor abstraction
  ↓
Vendor implementations
  ↓
External APIs
```

Each layer has a responsibility.

```
Controller
→ HTTP concerns

Service
→ business logic

Repository
→ persistence

PaymentProcessor
→ vendor-specific payment logic
```

This gives you:

- loose coupling
 - easier testing
 - easier replacement
 - separation of concerns
 - better maintainability
-

21. Dependency Injection + OOP

Spring makes these principles particularly useful.

Example:

```
@Service
public class PaymentService {

    private final PaymentProcessor processor;

    public PaymentService(PaymentProcessor processor) {
        this.processor = processor;
    }
}
```

The service depends on:

```
PaymentProcessor
```

not:

```
HydraPaymentProcessor
```

This is dependency inversion in practice.

Spring can inject the appropriate implementation.

So OOP + Dependency Injection work together:

```
Interface
  ↓
Abstraction
  ↓
Multiple implementations
  ↓
Polymorphism
  ↓
Dependency Injection
  ↓
Loose coupling
```

22. OOP vs SOLID

These are related but not identical.

OOP

Fundamental object-oriented concepts:

```
Encapsulation
Inheritance
Polymorphism
Abstraction
```

SOLID

Design principles for building maintainable OOP systems:

```
S → Single Responsibility
O → Open/Closed
L → Liskov Substitution
I → Interface Segregation
D → Dependency Inversion
```

For your interview, understand how they connect.

For example:

```
Abstraction + Polymorphism
  ↓
Open/Closed Principle

Interfaces + Dependency Injection
  ↓
Dependency Inversion Principle
```

23. Common Interview Traps

“Encapsulation means private variables.”

Incomplete.

`private` helps enforce encapsulation, but the real goal is:

Controlling access to state and enforcing invariants/behavior.

“Abstraction and encapsulation are the same.”

No.

Abstraction
→ expose essential behavior

Encapsulation
→ hide/protect internal state/implementation

“Inheritance is always good for code reuse.”

No.

Inheritance introduces coupling.

Prefer composition when the relationship is:

HAS-A

instead of:

IS-A

“Polymorphism only means method overloading.”

No.

For backend Java interviews:

Overloading → compile-time polymorphism
Overriding → runtime polymorphism

Runtime polymorphism is especially important in dependency injection and strategy-based designs.

“Interface is only for abstraction.”

Not exactly.

Interfaces also enable:

- polymorphism
- loose coupling
- dependency inversion
- multiple implementations
- testability

24. A Strong Interview Example

Suppose the interviewer asks:

“Tell me where you have used OOP principles in your project.”

A strong BPS-oriented answer would be:

In the payment processing flow, different payment vendors have vendor-specific implementations. We use a common abstraction for vendor processing, and the appropriate implementation is selected based on the vendor. This uses abstraction because the common contract hides vendor-specific details, and runtime polymorphism because the same interface can execute different vendor implementations. Encapsulation is used because each implementation keeps its provider-specific request construction, response handling, and validation internally. The Factory/Strategy design also avoids putting vendor-specific conditional logic throughout the payment service.

That is much stronger than:

“I used inheritance and polymorphism.”

25. OOP Cheat Sheet

ENCAPSULATION

↓

Protect state

Control how state changes

Example:

```
private status  
markPaid()
```

ABSTRACTION

↓

Expose contract

Hide implementation details

Example:

```
PaymentProcessor interface
```

INHERITANCE

↓
IS-A relationship
Reuse/extend behavior
Example:
PaymentException extends RuntimeException

POLYMORPHISM
↓
Same contract, different behavior
Example:
PaymentProcessor
├─ HydraProcessor
└─ ChaseProcessor

And in a real Spring Boot application:

Interface
↓
Abstraction
↓
Implementations
↓
Polymorphism
↓
Dependency Injection
↓
Loose coupling

26. Questions You Should Be Ready For

Fundamentals

1. What are the four pillars of OOP?
2. Explain encapsulation with a real example.
3. Encapsulation vs abstraction?
4. What is inheritance?
5. When should you avoid inheritance?
6. What is polymorphism?
7. Overloading vs overriding?
8. Compile-time vs runtime polymorphism?
9. Interface vs abstract class?
10. Composition vs inheritance?

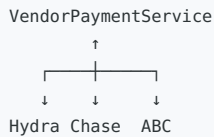
Backend design

11. How does polymorphism help in Spring dependency injection?
12. Why program to an interface?
13. How does Strategy Pattern use OOP?
14. How does Factory Pattern use OOP?
15. How does Dependency Injection relate to abstraction?
16. Give an example of OOP from your current project.
17. How would you add another payment vendor without changing existing vendor logic?
18. How would you make that code unit-testable?

Stronger 3-year question

“Suppose tomorrow BPS adds a new payment vendor, ABC. How would you design the code so existing Hydra and Chase logic doesn’t need to be heavily modified?”

A good direction is:



with vendor selection isolated in the Factory/Strategy mechanism.

That question tests **abstraction + polymorphism + Open/Closed Principle + dependency injection** together.